

Question 1:

>The running containers

Containers [Give feedback](#)

Container CPU usage

3.08% / 2800% (28 CPUs available)

Container memory usage

272.78MB / 15.08GB

Q Search

Only show running containers

	Name	Container ID	Image	Port(s)	CPU (%)	Last started
	answers	-	-	-	3.13%	2 hours ago
	annapurna-postgres	caebc66f1acb	postgres:16	5432:5432	3.08%	2 hours ago
	annapurna-minio	1d960cb45d71	minio/minio:latest	9000:9000	0.05%	2 hours ago

>Task A (Partitioning)

memory D

SELECT

store_id,

year,

month,

COUNT(*) AS row_count

FROM read_parquet(

's3://annapurna/curated/**/*.*parquet',

hive_partitioning=true

)

WHERE store_id = 'S07'

AND year = 2024

AND month = '07'

GROUP BY store_id, year, month;

store_id	year	month	row_count
varchar	int64	varchar	int64
S07	2024	07	5380

memory D

|

<

annapurna / curated / store_id=S01 / year=2024

▲ Name

month=01

month=02

month=03

month=04

month=05

month=06

month=07

month=08

month=09

month=10

month=11

month=12

```
memory D EXPLAIN ANALYZE
SELECT
  COUNT(*) AS row_count,
  ROUND(SUM(qty * unit_price), 2) AS amount
FROM read_parquet('annapurna_flat.parquet')
WHERE store_id = '587'
AND year = 2020
AND month = '07';
```

Query Profiling Information

EXPLAIN ANALYZE SELECT COUNT(*) AS row_count, ROUND(SUM(qty * unit_price), 2) AS amount FROM read_parquet('annapurna_flat.parquet') WHERE store_id = '587' AND year = 2020 AND month = '07';

HITRPS HTTP Stats

In: 0 bytes
out: 0 bytes
HEAD: 0
GET: 0
PUT: 0
POST: 0
DELETE: 0

Total Time: 0.0007s

QUERY

EXPLAIN_ANALYZE

0 rows
0.00s

PROJECTION

row_count
amount

1 row
0.00s

UNGROUPED_AGGREGATE

Aggregates:
count_star()
sum(qty)

1 row
0.00s

PROJECTION

(CAST(qty AS DOUBLE) *
unit_price)

5,380 rows
0.00s

TABLE_SCAN

Function:
READ_PARQUET
Projections:
qty
unit_price
Filters:
store_id='587'
month='07'
Total Files Read: 1
Filename(s):
annapurna_flat.parquet

5,380 rows
0.01s

>Task B(Idempotent loading)

No matter how many times you run you get the same curated dataset

```
memory D SELECT
COUNT(*) AS row_count,
SUM(hash(
  store_id, business_date, bill_no, line_no,
  product_code, qty, unit_price, line_type, ts
)) AS checksum
FROM read_parquet(
  's3://annapurna/curated/**/*.*.parquet',
  hive_partitioning=true
);
```

row_count int64	checksum int128
1120924	10338448750987504827944748

memory D |

>Task C (Dashboard dimension)

```
memory D DESCRIBE fact_sales_line;
```

fact_sales_line

store_id	varchar
business_date	timestamp
bill_no	varchar
line_no	varchar
product_code	varchar
product_sk	bigint
qty	bigint
unit_price	double
line_type	varchar
ts	timestamp
line_revenue	double

```
memory D |
```

```
memory D SELECT 'dim_store' AS table_name, COUNT(*) AS row_count
FROM dim_store
```

```
UNION ALL
```

```
SELECT 'dim_product', COUNT(*)
FROM dim_product
```

```
UNION ALL
```

```
SELECT 'dim_date', COUNT(*)
FROM dim_date;
```

table_name varchar	row_count int64
dim_store	12
dim_product	1224
dim_date	366

```
memory D SELECT
    s.store_id,
    s.store_name,
    ROUND(SUM(f.line_revenue), 2) AS net_revenue
FROM fact_sales_line f
JOIN dim_store s
    ON f.store_id = s.store_id
WHERE f.line_type IN ('SALE', 'RETURN', 'DISCOUNT', 'VOID')
GROUP BY s.store_id, s.store_name
ORDER BY net_revenue DESC;
```

store_id varchar	store_name varchar	net_revenue double
S03	Annapurna T Nagar	61864894.46
S10	Annapurna Rajouri Garden	59327041.98
S01	Annapurna Jayanagar	56995199.14
S06	Annapurna Andheri West	52584302.7
S02	Annapurna Koramangala	47194947.02
S12	Annapurna Salt Lake	44433598.82
S04	Annapurna Kukatpally	40201258.53
S07	Annapurna Baner	35219585.45
S08	Annapurna Satellite	34477161.43
S11	Annapurna Gomti Nagar	32429712.91
S05	Annapurna Kochi Marine	30159037.54
S09	Annapurna Vaishali	27978995.77

12 rows

3 columns

memory D |

```
memory D SELECT
    COALESCE(p.category_name, 'Unmapped / Non-product') AS category,
    ROUND(SUM(f.line_revenue), 2) AS net_revenue
FROM fact_sales_line f
LEFT JOIN dim_product p
    ON f.product_sk = p.product_sk
WHERE f.line_type IN ('SALE', 'RETURN', 'DISCOUNT', 'VOID')
GROUP BY 1
ORDER BY net_revenue DESC;
```

category varchar	net_revenue double
Staples & Grains	98677244.71
Baby Care	88746237.02
Edible Oils	82736636.78
Dairy	39696588.61
Personal Care	37437446.74
Frozen & Ready to Eat	29765097.34
Home Care	28253823.0
Beverages	25805866.86
Stationery & General	25749321.35
Spices & Masala	18408992.48
Confectionery	17251020.23
Fruits & Vegetables	15314853.8
Bakery	12957549.04
Biscuits & Snacks	7335274.17
Unmapped / Non-product	-5270216.38

15 rows

2 columns

memory D |

>Task D (march month audit logs)

memory D

SELECT
COUNT(*) AS audited_rows,
COUNT(*) FILTER (
WHERE price_difference <> 0
) AS differing_rows,
ROUND(SUM(ABS(price_difference)), 2)
AS total_absolute_unit_price_difference,
ROUND(SUM(
CASE
WHEN price_difference <> 0
THEN qty * price_difference
ELSE 0
END
) , 2) AS quantity_weighted_difference
FROM march_price_audit;

audited_rows	differing_rows	total_absolute_unit_price_difference	quantity_weighted_difference
int64	int64	double	double
62636	907	22349.26	-236.79

memory D |

memory D

SELECT
COUNT(*) AS march_price_revisions,
COUNT(DISTINCT product_sk) AS products_with_revisions,
MIN(effective_from) AS first_effective_date,
MAX(effective_from) AS last_effective_date
FROM pg.public.price_revisions
WHERE effective_from < DATE '2024-04-01'
AND (effective_to IS NULL
OR effective_to >= DATE '2024-03-01');

march_price_revisions	products_with_revisions	first_effective_date	last_effective_date
int64	int64	date	date
1377	1221	2022-01-01	2024-03-31

memory D |

>Task e (query across two systems minio and postgres)

memory D

SELECT
s.store_id,
st.store_name,
COUNT(*) AS line_count,
ROUND(SUM(s.qty * s.unit_price), 2) AS net_revenue
FROM read_parquet(
's3://annapurna/curated/**/*.parquet',
hive_partitioning=true
) s
JOIN pg.public.stores st
ON s.store_id = st.store_id
WHERE s.line_type IN ('SALE', 'RETURN', 'DISCOUNT', 'VOID')
GROUP BY s.store_id, st.store_name
ORDER BY s.store_id;

EXPLAIN ANALYZE SELECT
s.store_id, st.store_name, COUNT(*) AS line_count, ROUND(SUM(s.qty * s.unit_price), 2) AS net_revenue FROM read_parquet(
ve_partitioning=true) s JOIN pg.public.stores st ON s.store_id = st.store_id WHERE s.line_type IN ('SALE', 'RETURN', 'DISCOUNT', 'VOID') GROUP BY s.store_id, st.store_name ORDER BY s.store_id;

Query Profiling Information

HTTPFS HTTP Stats
in: 36.1 KiB
out: 0 bytes
#HEAD: 0
#GET: 1
#PUT: 0
#POST: 0
#DELETE: 0

Total Time: 0.0555s

QUERY
EXPLAIN_ANALYZE
0 rows
0.00s
ORDER_BY
s.store_id ASC



>>Task F (Reconcile)

```
memory D SELECT
    month,
    finance_revenue,
    folder_revenue,
    difference,
    classification
FROM monthly_reconciliation
ORDER BY month;
```

month varchar	finance_revenue double	folder_revenue double	difference double	classification varchar
2024-01	38446071.33	38446071.33	0.0	Matched
2024-02	34887085.55	34887085.55	0.0	Matched
2024-03	42457899.09	41971649.09	486250.0	Institutional invoice outside till
2024-04	37958457.37	37958457.37	0.0	Matched
2024-05	41764716.4	41764716.4	0.0	Matched
2024-06	38987082.82	38987082.82	0.0	Matched
2024-07	40527291.81	40295160.11	232131.7	Missing 507 files: 2024-07-09 to 2024-07-11
2024-08	45252181.75	45252181.75	0.0	Matched
2024-09	44615037.46	44615037.46	0.0	Matched
2024-10	56359195.92	56359195.92	0.0	Matched
2024-11	51583838.47	51583838.47	0.0	Matched
2024-12	50745209.0	50745259.48	-50.48	Per-bill rounding in finance

12 rows 5 columns

FINAL FILE STRUCTURE

- └─ build_curated.py
- └─ docker-compose.yml
- └─ inspect_sales.py
- └─ upload_curated.py
- └─ upload_sales.py